Application of trees

Application of trees

- We can solve different problems using trees.

1. How should items in a list be stored so that an item can be easily located? To solve these problems, we use the concept of binary search trees.
2. What series of decisions should be made to find an object with a certain property in a collection of objects of a certain type? To solve these problems, we use the concept of decision trees.
3. How should a set of characters be efficiency coded by bit strings? To solve these problems, we use the concept of prefix codes.

Application of Graphs

- Graphs are used to represent networks of communication, data organization, computational devices, flow of computation.
- Graphs can be used to show the link structures of web sites. The vertices are the web page available at the website and a directed edge from page A to page B exists if and only if A contains a link to B.
- Graph theory is also used to study molecules in chemistry and physics.
- In chemistry a graph makes a natural model for a molecule, where vertices represent atoms and edges bonds. This approach especially used in computer processing of molecular structures, ranging from chemical editors to database searching.

Binary search Algorithm

- A binary search algorithm(BST) is a binary tree in which each child of a vertex is designated as a right or left child, no vertex has more than one right child or left child and each vertex is labeled with a key, which is one the items.

- Furthermore, vertices are assigned keys so that the key of a vertex is both larger than the keys of all vertices in its left subtree and smaller than the keys of all vertices in its right subtree.

- We use a recursive procedure to form a binary search tree for a list of items.
- We start with a tree containing just one vertex, namely, the root.
- The first item in the list is assigned as the key of the root.
- To add a new item, first compare it with the keys of vertices already in the tree, starting at the root and moving to the left if the item is less than the key of respective vertex if this vertex has a left child or moving to the right if the item is greater than the key of the respective vertex if this vertex has a right child.
- When the item is less than the respective vertex and this vertex has no left child, then a new vertex with this item as its key is inserted as a new left child.
- Similarly, when the item is greater than the respective vertex and this vertex has no right child, then a new vertex with this item as its key is inserted as a new right child.

Question: Form a binary search tree for the words *mathematics, physics, geography, zoology, meteorology, geology, psychology*, and *chemistry* (using alphabetical order).

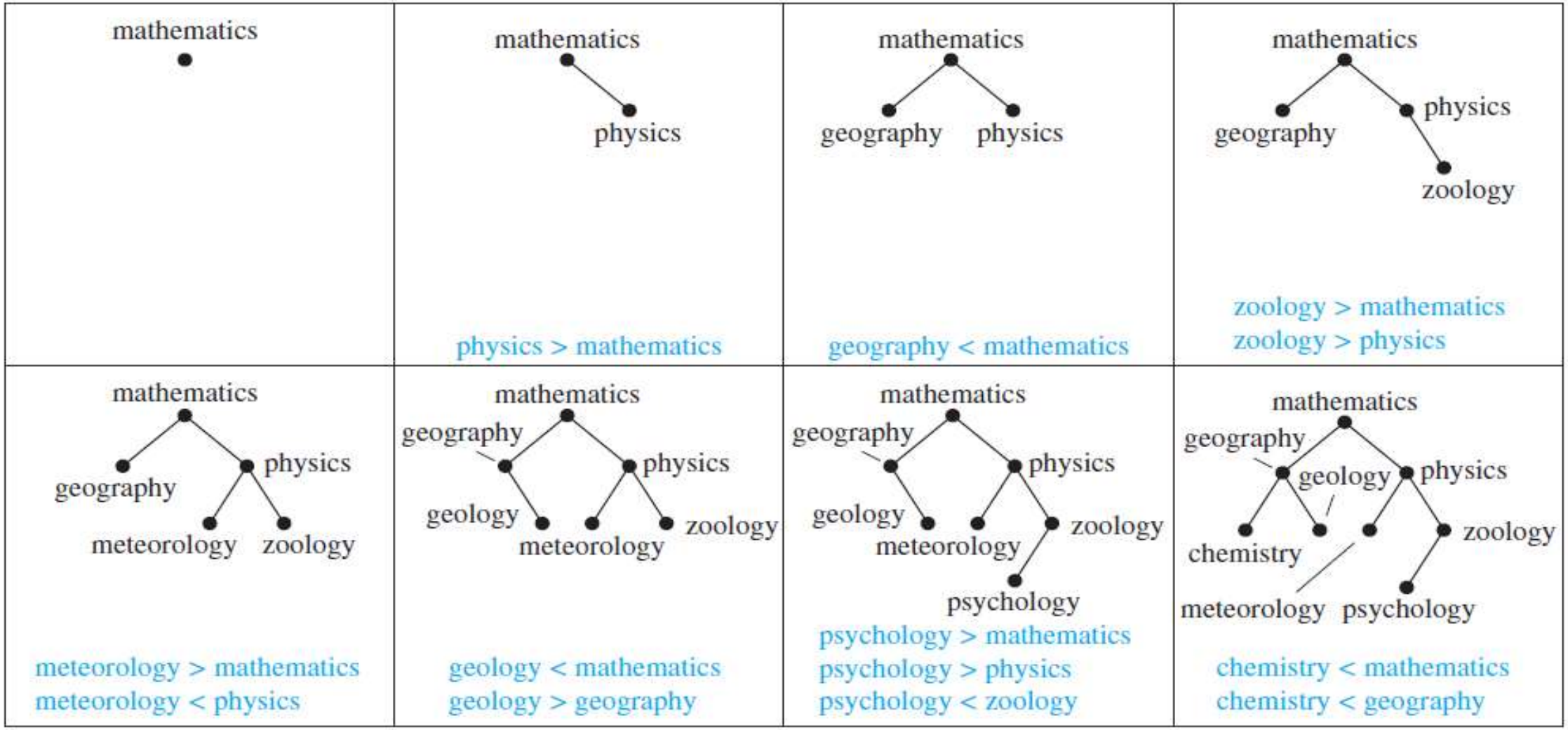


FIGURE 1 **Constructing a Binary Search Tree.**

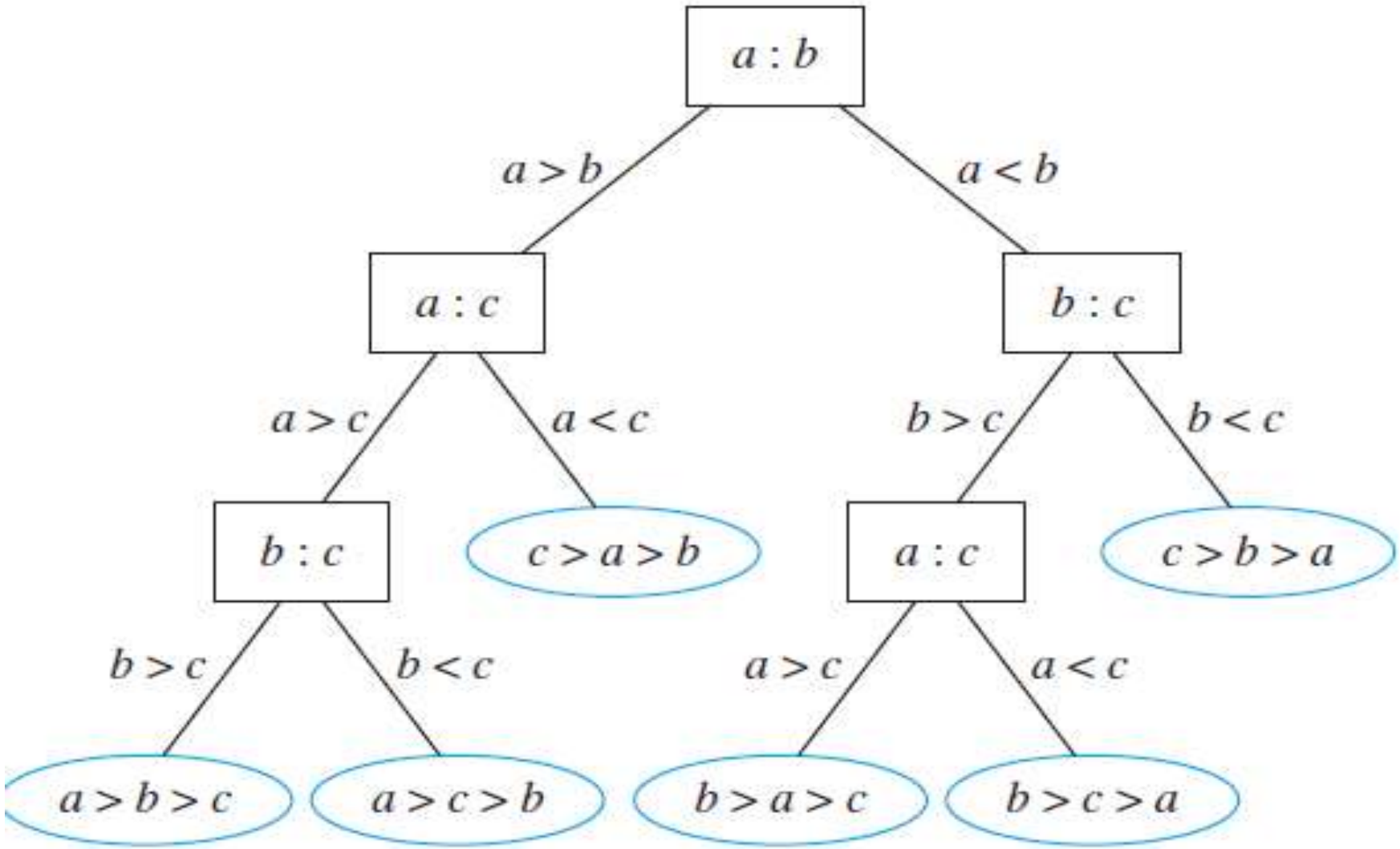
Assignment

- Build a binary search tree for the words oenology, phrenology, campanology, ornithology, ichthyology, limnology, alchemy, and astrology using alphabetic order.

Decision Tree

- Binary trees can be used to locate items based on comparisons, in this situation each comparison tells us to visit either left subtree or right subtree.
- A rooted tree where internal vertices correspond to a decision and subtree at these vertices gives possible outcomes of the decisions made is called decision tree.
- Example: Decision Tree for sorting three distinct Elements

- You know many of the sorting algorithms. In most of those algorithms the basic idea for sorting is comparison between two items at a time. In this situation those comparisons can be pictorially represented as full binary tree. Since we know that for n elements there may be n! solution that could be obtained from the comparison, we can say that there will be n! leaves in the formed decision tree.


a : b
a > b
a < b
a : c
b : c
a > c
a < c
b > c
b < c
b : c
c > a > b
a : c
c > b > a
b > c
b < c
a > c
a < c
a > b > c
a > c > b
b > a > c
b > c > a

Prefix Codes

- Consider the problem of using bit strings to encode the letters of the English alphabet.
- We can represent each letter with a bit string of length five, because there are only 26 letters and there are 32 bit strings of length five.
- The total number of bits used to encode data is five times the number of characters in the text when each character is encoded with five bits.
- Is it possible to find a coding scheme of these letters such that, when data are coded, fewer bits are used? We can save memory and reduce transmittal time if this can be done.

- Consider using bit strings of different lengths to encode letters. Letters that occur more frequently should be encoded using short bit strings, and longer bit strings should be used to encode rarely occurring letters.
- When letters are encoded using varying numbers of bits, some method must be used to determine where the bits for each character start and end.
- For instance, if *e* were encoded with 0, *a* with 1, and *t* with 01, then the bit string 0101 could correspond to *eat, tea, eaea*, or *tt.*
- One way to ensure that no bit string corresponds to more than one sequence of letters is to encode letters so that the bit string for a letter never occurs as the first part of the bit string for another letter.
- Codes with this property are called **prefix codes**.
- For instance, the encoding of *e* as 0, *a* as 10, and *t* as 11 is a prefix code. A word can be recovered from the unique bit string that encodes its letters.
- For example, the string 10110 is the encoding of *ate*. The initial 1 does not represent a character, but 10 does represent *a* (and could not be the first part of the bit string of another letter). Then, the next 1 does not represent a character, but 11 does represent *t*. The final bit, 0, represents *e*.

- A prefix code can be represented using a binary tree
- where the characters are the labels of the leaves in the tree. The edges of the tree are labeled so that an edge leading to a left child is assigned a 0 and an edge leading to a right child is assigned a 1.
- Characters are encoded with the bit string constructed using the labels of the edges in the unique path from the root to the leaves.

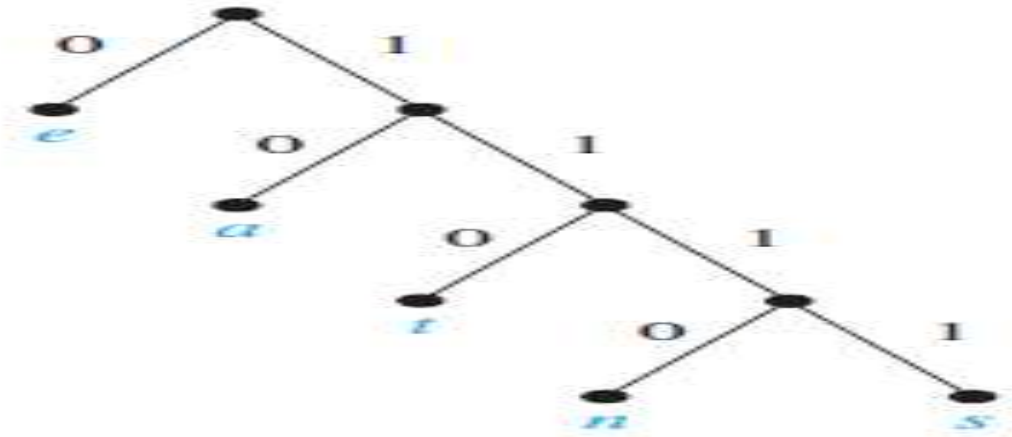


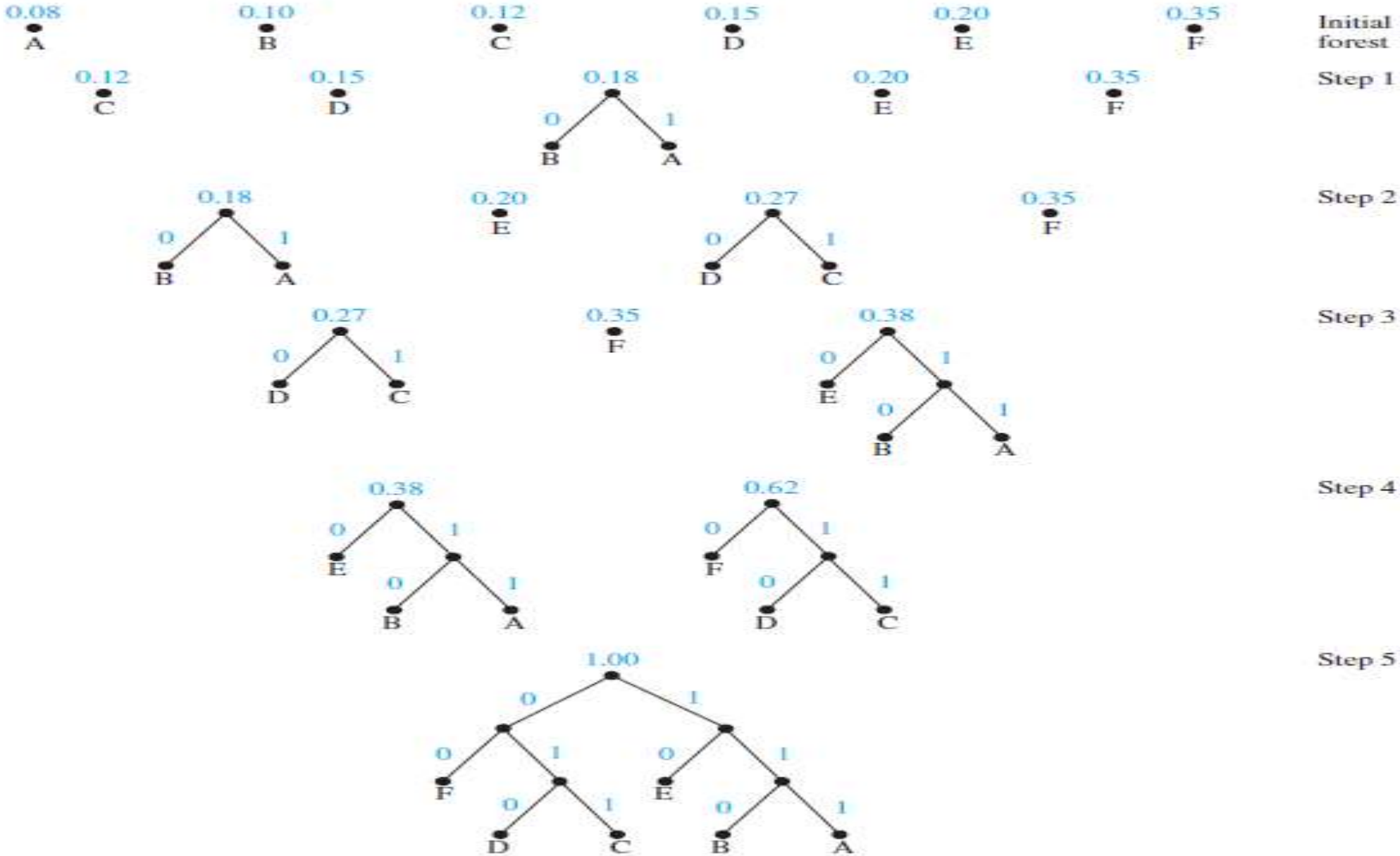
FIGURE 5 A Binary Tree with a Prefix Code.

ALGORITHM 2 Huffman Coding.

procedure *Huffman*(C: symbols a_i with frequencies w_i, $i = 1, \ldots, n$)
F := forest of n rooted trees, each consisting of the single vertex a_i and assigned weight w_i
while F is not a tree
  Replace the rooted trees T and T' of least weights from F with $w(T) \geq w(T')$ with a tree having a new root that has T as its left subtree and T' as its right subtree. Label the new edge to T with 0 and the new edge to T' with 1.
  Assign $w(T) + w(T')$ as the weight of the new tree.
{the Huffman coding for the symbol a_i is the concatenation of the labels of the edges in the unique path from the root to the vertex a_i}

- Use Huffman coding to encode the following symbols with the frequencies listed: A: 0.08, B: 0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35. What is the average number of bits used to encode a character?
- Answer: Figure displays the steps used to encode these symbols. The encoding produced encodes A by 111, B by 110, C by 011, D by 010, E by 10, and F by 00. The average number of bits used to encode a symbol using this encoding is

$$3 \cdot 0.08 + 3 \cdot 0.10 + 3 \cdot 0.12 + 3 \cdot 0.15 + 2 \cdot 0.20 + 2 \cdot 0.35 = 2.45$$


0.08
A
0.10
B
0.12
C
0.15
D
0.20
E
0.35
F
Initial forest
0.12
C
0.15
D
0.18
0
1
B
A
0.20
E
0.35
F
Step 1
0.18
0
1
B
A
0.20
E
0.27
0
1
D
C
0.35
F
Step 2
0.27
0
1
D
C
0.35
F
0.38
0
1
E
0
1
B
A
Step 3
0.38
0
1
E
0
1
B
A
0.62
0
1
F
0
1
D
C
Step 4
1.00
0
1
0
1
0
1
F
0
1
E
0
1
D
C
B
A
Step 5